Carinator - Car Database Management System

A sleek and simple web-based application for browsing, favoriting, and reviewing cars.

Project Structure

```
carinator/
├── frontend/
│   └── index.html         # Frontend application
├── backend/
│   └── app.py             # Flask API server
├── database/
│   └── carinator.db       # SQLite database (auto-created)
└── README.md              # This file
```

Features

- **User Authentication**: Register and login system
- **Browse Cars**: Explore cars with detailed information
- **Filtering**: Filter by brand and features using checkboxes
- **Favorites**: Mark cars as favorites for quick access
- **Notes**: Add multiple notes to any car
- **Reviews**: Write one review per car
- **Pre-populated Data**: Comes with sample cars, brands, and features

Prerequisites

- Python 3.7 or higher
- pip (Python package manager)
- A modern web browser (Chrome, Firefox, Safari, Edge)

Installation

1. Install Python Dependencies

bash

```
pip install flask flask-cors
```

2. Set Up the Project Structure

Create the following folder structure:

```
carinator/
├── frontend/
├── backend/
└── database/
```

- Save `index.html` in the `frontend/` folder
- Save `app.py` in the `backend/` folder
- The `database/` folder will auto-populate when you run the backend

Running the Application

Step 1: Start the Backend Server

Open a terminal, navigate to the backend folder, and run:

bash

```
cd backend
python app.py
```

You should see:

```
Starting Carinator Backend...
Database located at: ../database/carinator.db
Backend running on http://localhost:5000
```

Keep this terminal window open! The backend must be running for the application to work.

Step 2: Open the Frontend

Open the `frontend/index.html` file in your web browser:

- **Option 1**: Double-click the file
- **Option 2**: Right-click → Open with → Your browser
- **Option 3**: Drag and drop the file into your browser

Using the Application

1. Register/Login

- First time: Click "Register" to create an account
  - Enter username, email, and password
  - Click "Register"
- Returning users: Enter username and password, click "Login"

2. Explore Tab

- **Browse all cars** in the database
- **Filter by Brand**: Select one or more brands using checkboxes
- **Filter by Features**: Select features (cars must have ALL selected features)
- Click "Apply Filters" to see filtered results
- Each car card shows:
  - Car name, brand, model, year, country
  - Description
  - Feature tags
  - Action buttons (Favorite, Note, Review)

3. Favorites Tab

- View all cars you've marked as favorites
- Click the heart button to favorite/unfavorite a car

4. Notes Tab

- View all your notes organized by car
- Add multiple notes per car
- Edit or delete existing notes

5. Reviews Tab

- View all your reviews
- Add one review per car
- Edit or delete your reviews

Sample Data

The application comes pre-populated with:

Brands

- Toyota (Japan)
- Ford (USA)
- BMW (Germany)
- Ferrari (Italy)
- Tesla (USA)
- Mercedes-Benz (Germany)
- Honda (Japan)
- Porsche (Germany)

Features

- Electric
- Hybrid
- AWD (All-Wheel Drive)
- Turbo
- Autopilot
- Sunroof
- Leather Seats
- Navigation
- Bluetooth
- Backup Camera

Sample Cars

- Toyota Camry, Corolla
- Ford Mustang, F-150
- BMW 3 Series, X5
- Ferrari 488 GTB
- Tesla Model 3, Model Y
- Mercedes-Benz S-Class
- Honda Civic, Accord
- Porsche 911

Database Schema

Tables

1. **users -** User accounts
2. **brands -** Car manufacturers
3. **cars -** Vehicle information
4. **features -** Car features
5. **car_features -** Links cars to features (many-to-many)
6. **favorites -** User's favorited cars
7. **notes -** User notes on cars (multiple per car)
8. **reviews -** User reviews (one per car per user)

API Endpoints

Authentication

- `POST /api/register` - Register new user

- `POST /api/login` - Login user

Brands & Features

- `GET /api/brands` - Get all brands
- `GET /api/features` - Get all features

Cars

- `GET /api/cars` - Get all cars (with optional filters)
- `GET /api/cars/<car_id>` - Get specific car

Favorites

- `GET /api/favorites/<user_id>` - Get user's favorites
- `POST /api/favorites` - Add favorite
- `DELETE /api/favorites` - Remove favorite

Notes

- `GET /api/notes/user/<user_id>` - Get user's notes
- `POST /api/notes` - Create note
- `PUT /api/notes/<note_id>` - Update note
- `DELETE /api/notes/<note_id>` - Delete note

Reviews

- `GET /api/reviews/user/<user_id>` - Get user's reviews
- `GET /api/reviews/<user_id>/<car_id>` - Get specific review
- `POST /api/reviews` - Create/update review
- `DELETE /api/reviews/<user_id>/<car_id>` - Delete review

Troubleshooting

Backend won't start

Error: `ModuleNotFoundError: No module named 'flask'`

- **Solution**: Install Flask: `pip install flask flask-cors`

Frontend can't connect to backend

Error: "Connection error. Make sure the backend is running."

- **Solution**: Make sure the backend is running on `http://localhost:5000`
- Check that no other application is using port 5000

Port 5000 already in use

Error: `Address already in use`

- **Solution**: Change the port in `app.py` (last line):

python

```
app.run(debug=True, port=5001)  # Change to 5001 or any free port
```

- Also update the frontend's `API_URL` in `index.html`:

javascript

```
const API_URL = 'http://localhost:5001/api';
```

Database issues

Error: Database-related errors

- **Solution**: Delete `database/carinator.db` and restart the backend
- The database will be recreated with fresh sample data

Future Enhancements (Admin Panel)

As mentioned, you'll be creating an admin panel later to:

- Add new cars to the database
- Remove cars based on user requests
- Update car information
- Manage brands and features

This will be a separate Python script in the `database/` folder.

Notes

- The application uses localStorage to persist login sessions
- Passwords are hashed using SHA-256 (for production, use bcrypt)
- CORS is enabled for local development
- Database is created automatically on first run

Support

For issues or questions, refer to the code comments or modify as needed for your specific requirements.

Happy car browsing! 🚙